Kubernetes alkalmazások konfigurációja

Back István Levente

Balogh Efraim

Lazar Emanuel

Mikroszervíz architektúrák 2026

Miről lesz szó?

1. **ConfigMap** - nem érzékeny konfiguráció kezelése
2. **Secret** - titkos adatok natív K8s megoldása
3. **External Secrets Operator + Vault** - éles környezeti megoldás

> **A 12-factor app III. alapelve:**
> *"Store config in the environment"* - válaszd szét a kódot a konfigurációtól.

Miért fontos a konfiguráció szétválasztása?

- Ugyanaz az image fusson **dev / staging / prod** környezetben
- Ne kelljen újraépíteni az image-et csak azért, mert egy érték változik
- A titkok **soha** ne kerüljenek a kódba vagy a Git-be
- A környezet-specifikus értékek **deklaratívan** legyenek kezelve
- **GitOps**-barát: a konfiguráció verziózható, auditálható

1. rész

ConfigMap

Mi az a ConfigMap?

> **Egy mondat:** Namespace-scoped key-value tár nem érzékeny konfigurációhoz.

- Kulcs-érték párok tárolása a klaszteren belül
- **Nem titkosított** - bárki olvashatja, akinek RBAC joga van rá
- Pod-ok env változóként vagy fájlként érik el
- Maximum **1 MiB** méretkorlát

Milyen problémát old meg?

ConfigMap előtt:

- Konfiguráció a Docker image-be építve → újraépítés minden változásnál
- Hardcoded értékek a kódban
- Konfiguráció környezet-specifikus build-ekben

ConfigMap után:

- Egy image, sok környezet
- Deklaratív, verziózható konfig
- Futás közben módosítható (volume mount-tal)

Előnyök / Hátrányok

Előnyök:

- Feature flagek
- Log szintek
- API endpoint URL-ek
- App-specifikus paraméterek
- Nginx konfig fájlok
- Environment azonosítók

Hátrányok:

- Jelszavak, API tokenek
- TLS tanúsítványok
- 1 MiB-nál nagyobb fájlok
- Bináris adatok (kerülendő)
- Bármilyen érzékeny adat

Hogyan működik belül?

1. ConfigMap **etcd-ben** tárolódik plaintext-ben
2. Pod létrehozásakor a **kubelet** lekéri a tartalmát
3. A pod-ba **kétféleképpen** juthat el:
    - **Environment változó** - pod indulásakor egyszer (statikus)
    - **Volume mount** - fájlrendszerre projektálva, **élőben frissül**
4. Volume mount esetén a kubelet ~30-60 másodpercenként szinkronizál

ConfigMap létrehozása

CLI-ből:

```
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info
```

YAML-ból:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: info
```

Pod-hoz csatolás - env változó

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: demo-pod
spec:
  containers:
    - name: demo
      image: alpine:3.19
      envFrom:
        - configMapRef:
            name: app-config
```

Figyelem: A pod nem látja az új értékeket módosítás után - restart kell!

Pod-hoz csatolás - volume mount

```yaml
spec:
  containers:
    - name: demo
      image: alpine:3.19
      volumeMounts:
        - name: config-vol
          mountPath: /etc/config
  volumes:
    - name: config-vol
      configMap:
        name: app-config
```

~30 mp után automatikusan frissül, restart nélkül!

Csatolási módok összehasonlítása

Mód	Hot reload	Használat	Mire való
env változó	❌	`process.env.LOG_LEVEL`	Statikus konfig
envFrom	❌	Összes kulcs env-ként	Sok érték együtt
volume mount	✅	`/etc/config/LOG_LEVEL` fájl	Dinamikus konfig
subPath mount	❌	Egy fájl mountja	Egyedi fájl

Biztonság és limitációk

- **Nincs titkosítás** - etcd-ben sima szöveg
- **1 MiB** méretkorlát
- `kubectl get configmap` mindenkinek látható, akinek RBAC joga van
- **Immutable** flag (K8s 1.19+) - utána nem módosítható
- Volume mount **subPath**-szal **nem** frissül automatikusan

Alternatívák

- **Helm values** - chart-szintű konfigurációkezelés
- **Kustomize overlays** - környezet-specifikus felülírások
- **External config providers** - pl. Spring Cloud Config
- **CRD-alapú konfig** - saját Custom Resource-ok

DEMO 1

ConfigMap

2. rész

Secret

Mi az a Secret?

> **Egy mondat:** Namespace-scoped key-value tár érzékeny adatokhoz, base64-kódolva tárolva.

- Hasonló a ConfigMap-hez, de **érzékeny adatokra** szánt
- Az értékek **base64-kódoltak** (nem titkosítottak!)
- Több típus: `Opaque`, `kubernetes.io/tls`, `dockerconfigjson`, stb.
- A pod-ok ugyanúgy érik el, mint a ConfigMap-et

Milyen problémát old meg?

Secret előtt:

- Jelszavak ConfigMap-ben (nincs típuskülönbség)
- Hardcoded titkok a kódban / image-ben
- TLS tanúsítványok ad-hoc kezelése

Secret után:

- Külön típus érzékeny adatokra → tooling tudja kezelni
- Memóriában (tmpfs) mountolva, nem lemezen
- RBAC-cel külön kezelhető

ConfigMap vs Secret — mikor mit?

Adat típusa	ConfigMap	Secret
Log szint, feature flag	✅	❌
API endpoint URL	✅	❌
Környezet neve (dev/prod)	✅	❌
Adatbázis jelszó	❌	✅
API kulcs (Stripe, SendGrid)	❌	✅
TLS tanúsítvány + privát kulcs	❌	✅
Docker registry hitelesítés	❌	✅
OAuth client secret	❌	✅

Valós életbeli példák

Mit tárolunk Secret-ben a gyakorlatban?

- **Adatbázis hitelesítés** — PostgreSQL / MySQL / MongoDB jelszó, connection string
- **Külső API kulcsok** — Stripe fizetési kulcs, SendGrid e-mail API token, Twilio SID
- **TLS tanúsítvány** — HTTPS-hez szükséges cert + privát kulcs (`kubernetes.io/tls`)
- **Image pull secret** — privát Docker / GitHub Container Registry hitelesítés
- **JWT aláíró kulcs** — token generáláshoz / validáláshoz használt titkos kulcs
- **OAuth client secret** — Google, GitHub, Microsoft SSO integráció
- **SSH privát kulcs** — Git műveletek, szerver hozzáférés automation esetén

> Mindezek közös tulajdonsága: **kiszivárgásuk közvetlen biztonsági kockázatot jelent.**

Előnyök / Hátrányok

Előnyök (kis projekteknél):

- Adatbázis jelszó dev környezetben
- TLS tanúsítvány tárolása
- Image pull credentials
- Egyszerű API tokenek

Hátrányok (éles környezetben önmagában):

- Multi-cluster setup
- Auditált hozzáférés
- Automatikus rotáció
- Dinamikus credentialek

A nagy félreértés: base64 ≠ titkosítás

```
$ echo "SuperSecret123!" | base64
U3VwZXJTZWNyZXQxMjMhCg==

$ echo "U3VwZXJTZWNyZXQxMjMhCg==" | base64 -d
SuperSecret123!
```

Figyelem: Bárki, akinek `kubectl get secret` joga van, olvashatja a titkokat.

Hogyan működik belül?

1. Secret szintén **etcd-ben** tárolódik
2. Alapból **NEM titkosított** - encryption at rest külön konfigurálandó
3. A pod-on **tmpfs**-en (memóriában) mountolódik
4. Csatolási módok ugyanazok mint ConfigMap-nél:
    - env változó (statikus)
    - volume mount (dinamikus)

Secret létrehozása

CLI-ből:

```
kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password='SuperSecret123!'
```

YAML-ból (base64-kódolva):

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4=
  password: U3VwZXJTZWNyZXQxMjMh
```

Secret típusok

Típus	Felhasználás
`Opaque`	Általános titok (default)
`kubernetes.io/tls`	TLS tanúsítvány + kulcs
`kubernetes.io/dockerconfigjson`	Docker registry hitelesítés
`kubernetes.io/service-account-token`	ServiceAccount token
`kubernetes.io/basic-auth`	HTTP basic auth
`kubernetes.io/ssh-auth`	SSH privát kulcs

Biztonság és limitációk

- **base64 ≠ titkosítás** - a legnagyobb félreértés
- **etcd plaintext** alapból (encryption at rest külön)
- `kubectl get secret` = klaszter szintű kompromisszum
- **Git-be nem rakható** plaintext-ben
- **Nincs natív rotáció** - manuálisan kell frissíteni
- **Nincs audit log** - ki, mikor olvasta?

Alternatívák a natív Secret-en túl

- **Sealed Secrets** (Bitnami) - GitOps-barát, encrypted Secret
- **SOPS + age/PGP** - Mozilla eszköz Git-encrypted titkokra
- **git-crypt** - egyszerű Git fájl-titkosítás
- **External Secrets Operator** ← *erről szól a 3. rész!*

DEMO 2

Secret

3. rész

External Secrets Operator + HashiCorp Vault

Miért nem elég a natív Secret?

A natív Secret hiányosságai éles környezetben:

- **Nincs titkosítás** alapból (etcd plaintext)
- **Nincs natív rotáció** - kézzel kell frissíteni
- **Nincs részletes audit log** - ki olvasta, mikor?
- **Multi-cluster fájdalom** - minden klaszterben külön
- **GitOps probléma** - nem tehető Git-be
- **Statikus credentialek** - nincs dinamikus, rövid életű hozzáférés

Mi az a külső secret manager?

> **Egy mondat:** Dedikált, központi rendszer titkos adatok tárolására, hozzáférés-vezérlésére, naplózására és rotációjára.

Ismertebb példák:

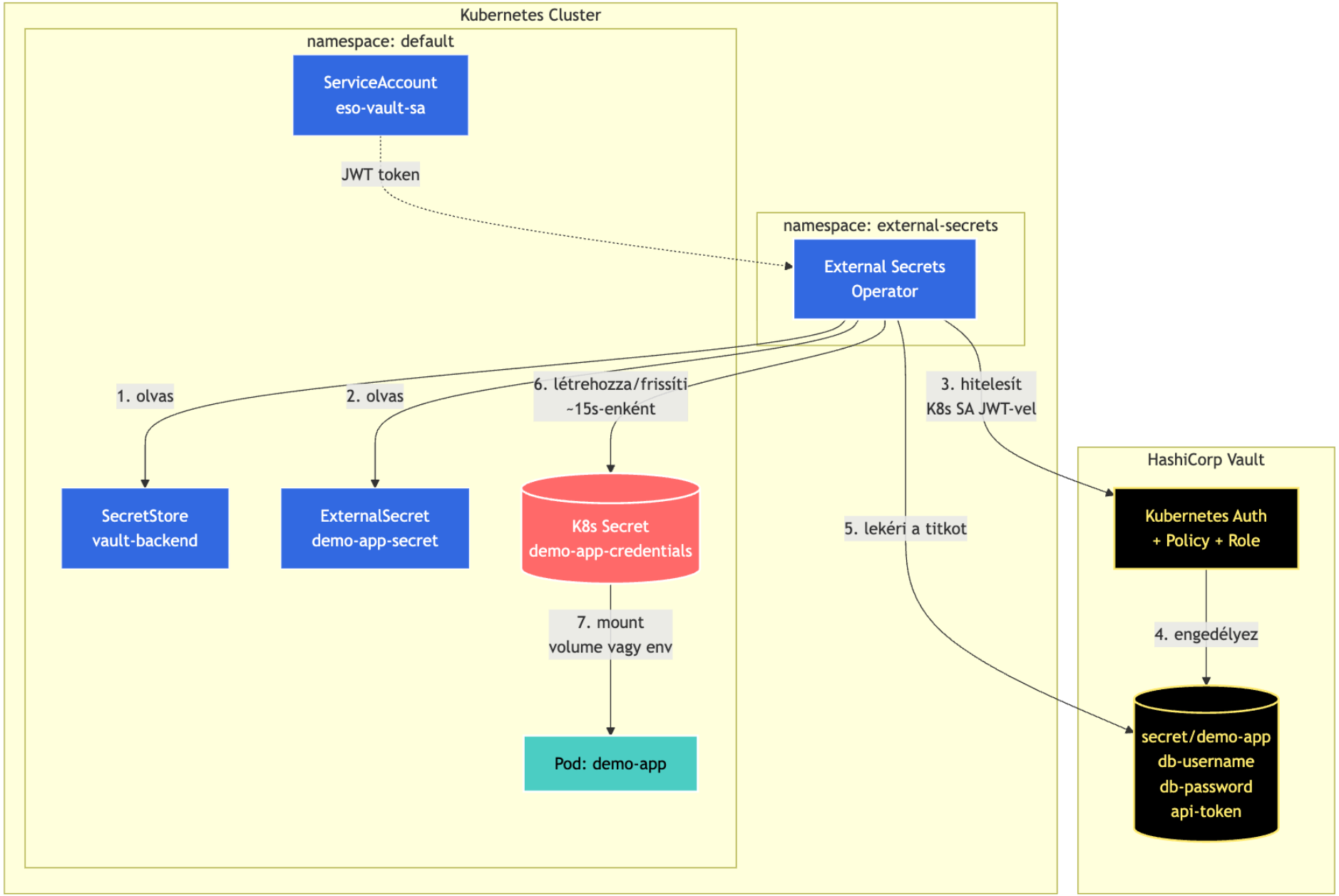
- **HashiCorp Vault** ← demónkban ezt használjuk
- **AWS Secrets Manager**
- **Azure Key Vault**
- **GCP Secret Manager**
- **Akeyless, CyberArk, IBM Cloud Secrets Manager**

Két különálló réteg

1. **Secret Store (a tényleges tár)**
   - Itt élnek a titkok titkosítva
   - Audit log, hozzáférés-vezérlés, rotáció
   - Pl. Vault, AWS Secrets Manager
2. **Integrációs réteg (ami a klaszterbe juttatja)**
   - **External Secrets Operator (ESO)** ← demónkban
   - **Secrets Store CSI Driver**
   - **Vault Agent Sidecar Injector**

Mi az az External Secrets Operator?

- **CNCF projekt**, vendor-független
- Egy K8s **operator**: a klaszterben fut, CRD-ket figyel
- Két fő Custom Resource:
  - `SecretStore` - kapcsolat a külső tárolóhoz
  - `ExternalSecret` - mit, honnan, hova szinkronizálni
- A pod-od egy **sima K8s Secret-et** lát - transzparencia!


Kubernetes Cluster
namespace: default
ServiceAccount
eso-vault-sa
JWT token
namespace: external-secrets
External Secrets
Operator
1. olvas
2. olvas
6. létrehozza/frissíti
~15s-enként
3. hitelesít
K8s SA JWT-vel
SecretStore
vault-backend
ExternalSecret
demo-app-secret
K8s Secret
demo-app-credentials
5. lekéri a titkot
7. mount
volume vagy env
Pod: demo-app
HashiCorp Vault
Kubernetes Auth
+ Policy + Role
4. engedélyez
secret/demo-app
db-username
db-password
api-token

Előnyök / Hátrányok

Előnyök:

- Adatbázis jelszavak, API tokenek
- TLS tanúsítványok
- Cloud provider kulcsok (AWS, GCP)
- **Dinamikus credentialek** (Vault egyik nagy ereje)
- Multi-cluster / multi-cloud

Hátrányok (vagy túlzás):

- Egyszerű feature flagek (oda ConfigMap)
- Lokális dev környezet
- Olyan projekt, ahol nincs csapat üzemeltetni

Hogyan működik belül?

1. ESO operator a klaszterben fut, figyeli a CRD-ket
2. `SecretStore` definiálja a kapcsolatot Vault felé
   - URL, hitelesítés (K8s ServiceAccount JWT)
3. `ExternalSecret` definiálja: melyik kulcsot, melyik K8s Secret-be
4. Reconciliation loop: ESO `refreshInterval`-onként szinkronizál
5. A Vault válasz alapján létrejön/frissül a K8s Secret

SecretStore példa

```yaml
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: vault-backend
spec:
  provider:
    vault:
      server: "http://vault.default.svc:8200"
      path: "secret"
      version: "v2"
      auth:
        kubernetes:
          mountPath: "kubernetes"
          role: "demo-app"
          serviceAccountRef:
            name: "eso-vault-sa"
```

ExternalSecret példa

```yaml
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: demo-app-secret
spec:
  refreshInterval: "15s"
  secretStoreRef:
    name: vault-backend
    kind: SecretStore
  target:
    name: demo-app-credentials
  data:
    - secretKey: password
      remoteRef:
        key: demo-app
        property: db-password
```

A "Secret zero" probléma

> **Mivel hitelesítünk a Vault felé? Ez maga is egy titok.**

Megoldások:

- **Kubernetes Auth** - ServiceAccount JWT (demónkban ez)
- **Workload Identity** - felhő-natív (GKE, EKS, AKS)
- **AppRole** - Vault-specifikus role-id + secret-id
- **Cloud IAM** - pl. AWS IAM role for ESO

Biztonság és limitációk

- **Network dependency** - ha Vault nem elérhető, új pod-ok nem indulnak
- **Cache vs frissesség** - `refreshInterval` egyensúly
- **RBAC a klaszterben is fontos** - generált Secret hozzáférhetősége
- **Komplexitás** - plusz komponens, plusz failure mode
- **Authentikáció** - secret zero probléma

Provider váltás - vendor-függetlenség

Vault helyett AWS Secrets Manager:

```yaml
spec:
  provider:
    aws:
      service: SecretsManager
      region: eu-central-1
      auth:
        jwt:
          serviceAccountRef:
            name: my-sa
```

> **Az `ExternalSecret` ugyanaz marad!** Csak a `SecretStore` provider-blokk változik.

Alternatív integrációk

Megoldás	Hogyan működik	Komplexitás
External Secrets Operator	Sync K8s Secret-té	Közepes
Secrets Store CSI Driver	Volume-ként mountol	Közepes
Vault Agent Sidecar	Sidecar minden pod mellé	Magas
Sealed Secrets	Encrypted Secret Git-ben	Alacsony

DEMO 3

External Secrets Operator + Vault

Összehasonlítás

A három megoldás összevetése

Szempont	ConfigMap	Secret	ESO + Vault
Érzékeny adat	❌	Részben (base64)	✅
Titkosítás	❌	Részben (etcd)	✅
Audit log	❌	❌	✅
Rotáció	manuális	manuális	✅
GitOps-barát	✅	❌	✅
Komplexitás	alacsony	alacsony	közepes
Multi-cluster	❌	❌	✅

Mikor mit használjunk?

- **ConfigMap** - minden nem érzékeny konfigurációhoz
- **Secret** - kis projektnél, vagy ha az ESO túlzás (pl. lokális dev)
- **ESO + külső manager** - éles környezet, közepes-nagy klaszter, compliance

> A három **nem alternatíva**, hanem **kiegészítő**.
> Egy éles app jellemzően mindhármat használja egyszerre.

Költség-haszon

ESO + Vault előnyei:

- Központosított titok-kezelés
- Audit, rotáció, dinamikus credentialek
- Vendor-független (provider váltható)

Hátrányai:

- Plusz komponensek (Vault + ESO)
- Plusz failure mode-ok
- Tanulási görbe
- Üzemeltetési overhead

Tanulságok

1. **Válaszd szét a kódot a konfigurációtól** - 12-factor alapelv
2. **Az érzékenységi szint határozza meg a tárolót** - ConfigMap / Secret / külső
3. **base64 ≠ titkosítás** - ezt sokan félreértik
4. **Secret zero probléma létezik** - a hitelesítés is titok
5. **Hot reload csatolási módfüggő** - env nem, volume igen
6. **Az ESO vendor-független** - provider lecserélhető YAML-szinten

Bibliográfia

- **Kubernetes hivatalos dokumentáció:** kubernetes.io/docs
- **External Secrets Operator:** external-secrets.io
- **HashiCorp Vault:** developer.hashicorp.com/vault
- **The Twelve-Factor App:** 12factor.net
- **CNCF Cloud Native Landscape** - Secret Management kategória

> **Demó forráskód publikus repón:** *(link a végén)*

Köszönjük a figyelmet!

Kérdések?

Back István Levente

Balogh Efraim

Lazar Emanuel

Mikroszervíz architektúrák 2026